Architecture 1001: x86-64 Assembly MulDivExample.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

MulDivExample.c

```c
#define uint64 unsigned long long
uint64 main(){
    uint64 a = 0xdefec7ed;
    a *= 0xde7ec7ab1e;
    a /= 0x2bad505ad;
    return a;
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  eax,0DEFEC7EDh
0000000140001009  mov  qword ptr [rsp],rax
000000014000100D  mov  rax,0DE7EC7AB1Eh
0000000140001017  mov  rcx,qword ptr [rsp]
000000014000101B  imul rcx,rax
000000014000101F  mov  rax,rcx
0000000140001022  mov  qword ptr [rsp],rax
0000000140001026  xor  edx,edx
0000000140001028  mov  rax,qword ptr [rsp]
000000014000102C  mov  rcx,2BAD505ADh
0000000140001036  div  rax,rcx
0000000140001039  mov  qword ptr [rsp],rax
000000014000103D  mov  rax,qword ptr [rsp]
0000000140001041  add  rsp,18h
0000000140001045  ret
```

MulDivExample.c

```c
#define uint64 unsigned long long
uint64 main(){
    uint64 a = 0xdefec7ed;
    a *= 0xde7ec7ab1e;
    a /= 0x2bad505ad;
    return a;
}
```

```
main:
0000000140001000  sub   rsp,18h
0000000140001004  mov   eax,0DEFEC7EDh
0000000140001009  mov   qword ptr [rsp],rax
000000014000100D  mov   rax,0DE7EC7AB1Eh
0000000140001017  mov   rcx,qword ptr [rsp]
000000014000101B  imul  rcx,rax
000000014000101F  mov   rax,rcx
0000000140001022  mov   qword ptr [rsp],rax
0000000140001026  xor   edx,edx
0000000140001028  mov   rax,qword ptr [rsp]
000000014000102C  mov   rcx,2BAD505ADh
0000000140001036  div   rax,rcx
0000000140001039  mov   qword ptr [rsp],rax
000000014000103D  mov   rax,qword ptr [rsp]
0000000140001041  add   rsp,18h
0000000140001045  ret
```

DIV - Unsigned Divide

- Three forms
  - Unsigned divide ax by r/m8, al = quotient, ah = remainder
  - Unsigned divide edx:eax by r/m32, eax = quotient, edx = remainder
  - Unsigned divide rdx:rax by r/m64, rax = quotient, rdx = remainder
- If dividend is 32/64bits, edx/rdx will just be set to 0 by the compiler before the instruction (as occurred in the MulDivExample.c code)
- If the divisor is 0, a divide by zero exception is raised.

initial

operation

result

ax	r/m8(cl)
0x8	0x3

div ax, cl

ah	al
0x2	0x2

rdx	rax	r/mX(rcx)
0x0	0x8	0x5

div rax, rcx

rdx	rax	r/mX(rcx)
0x3	0x1	0x5

IDIV - Signed Divide

- If you were to then change MulDivExample to signed, you would see the IDIV instruction appear
- Three forms
  - Signed divide ax by r/m8, al = quotient, ah = remainder
  - Signed divide edx:eax by r/mX, eax = quotient, edx = remainder
  - Signed divide rdx:rax by r/m64, rax = quotient, rdx = remainder
- If dividend is 32/64bits, edx/rdx will just be set to 0 by the compiler before the instruction
- If the divisor is 0, a divide by zero exception is raised.

initial

ax	r/m8(cl)
0xFE	0x2

operation: div ax, cl

result

ah	al
0x0	0xFF

rdx	rax	r/mX(rcx)
0x0	0x57e11a	0xb01d

div rax, rcx

rdx	rax	r/mX(rcx)
0x82B7	0x7F	0x3



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING

MulDivExample.c Takeaways

- When a multiply or divide is not by a power of 2, compilers will use normal multiply/divide instructions
- VS compiler prefers IMUL over MUL (unsigned multiply)

```
#define uint64 unsigned long long
uint64 main(){
    uint64 a = 0xdefec7ed;
    a *= 0xde7ec7ab1e;
    a /= 0x2bad505ad;
    return a;
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  eax,0DEFEC7EDh
0000000140001009  mov  qword ptr [rsp],rax
000000014000100D  mov  rax,0DE7EC7AB1Eh
0000000140001017  mov  rcx,qword ptr [rsp]
000000014000101B  imul rcx,rax
000000014000101F  mov  rax,rcx
0000000140001022  mov  qword ptr [rsp],rax
0000000140001026  xor  edx,edx
0000000140001028  mov  rax,qword ptr [rsp]
000000014000102C  mov  rcx,2BAD505ADh
0000000140001036  div  rax,rcx
0000000140001039  mov  qword ptr [rsp],rax
000000014000103D  mov  rax,qword ptr [rsp]
0000000140001041  add  rsp,18h
0000000140001045  ret
```

Instructions we now know (28)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP/TEST
- AND/OR/XOR/NOT
- INC/DEC
- SHR/SHL/SAR/SAL
- **DIV/IDIV**

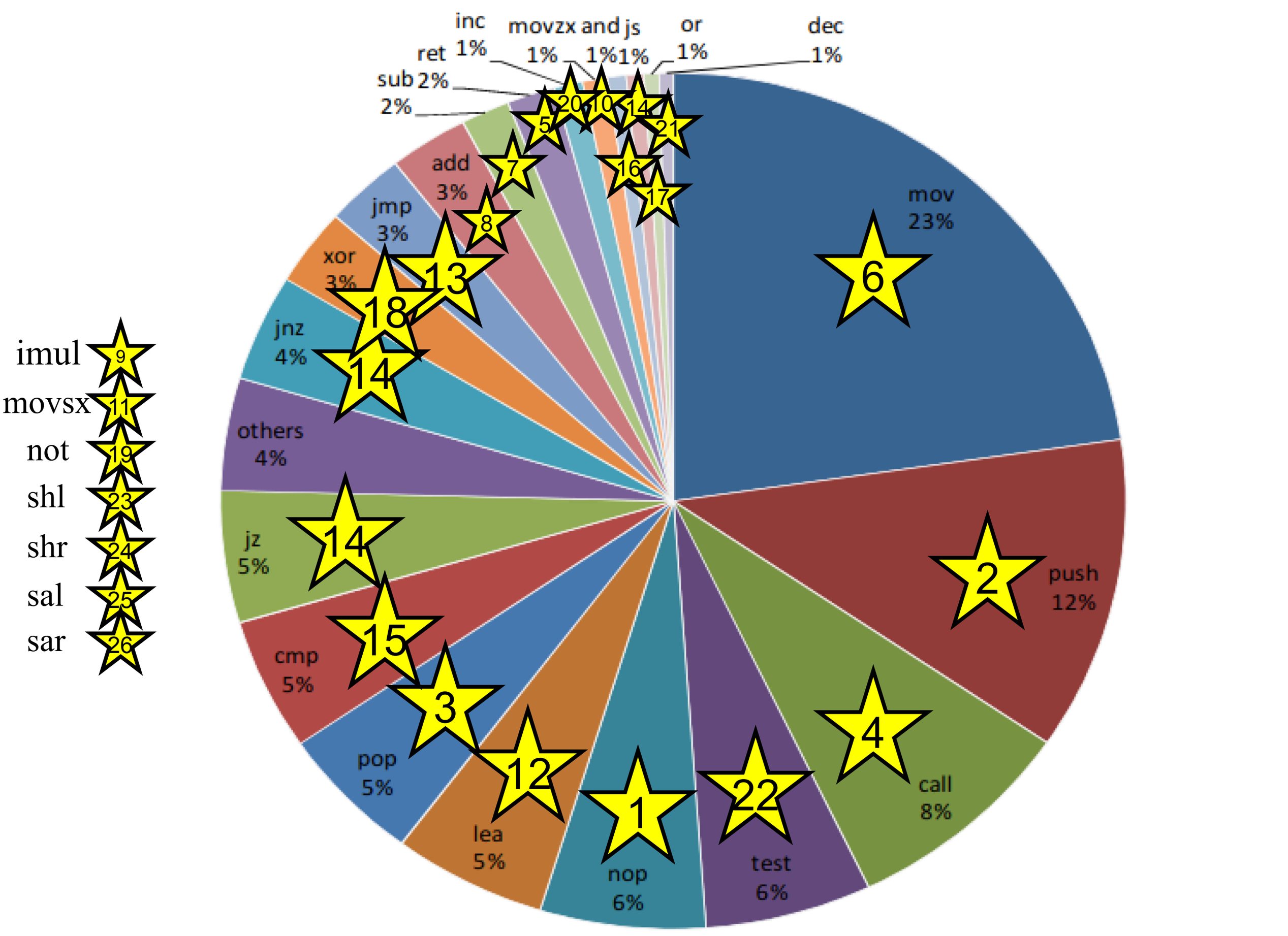


Fig. 2 Web browsers' instruction frequencies

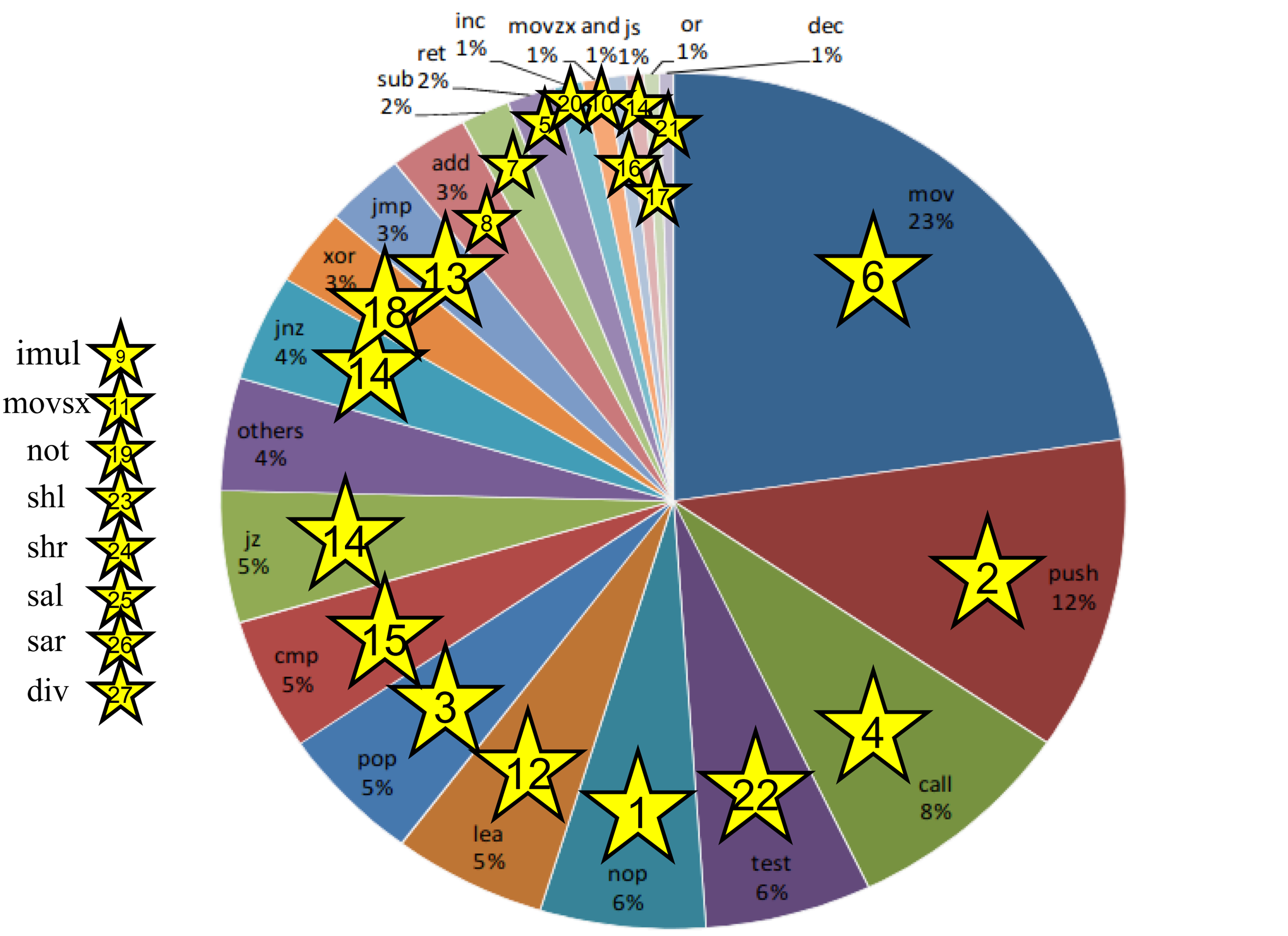


Fig. 2 Web browsers' instruction frequencies

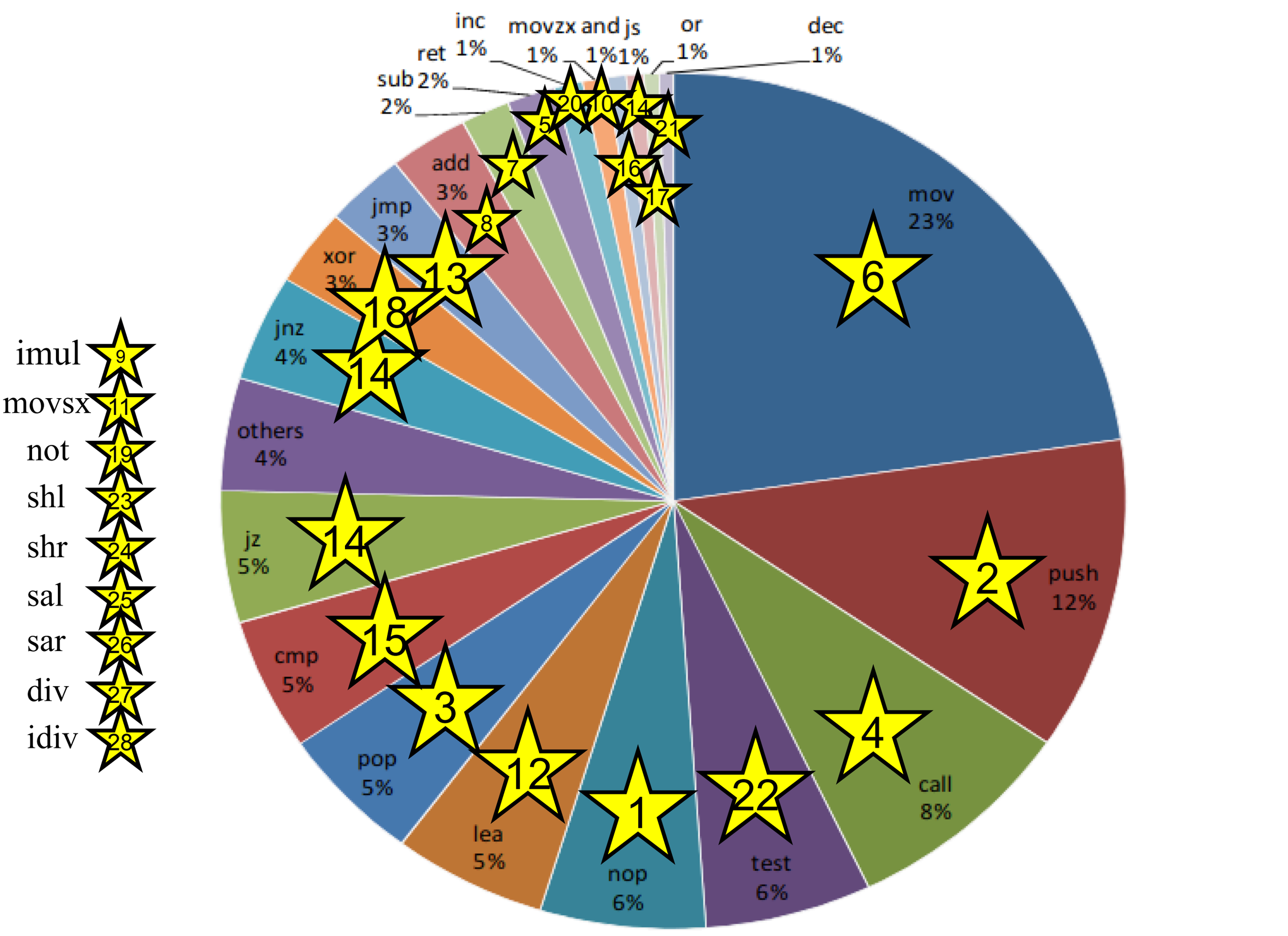


Fig. 2 Web browsers' instruction frequencies